

Decoupled Image-to-Video Generation via Latent Flow: An Investigation into Embedding-Conditioned Rendering

Anonymous COMP4471 submission

Paper ID

Abstract

001 *Image-to-video generation from a single static image and*
002 *an action label requires simultaneously understanding both*
003 *what is in the scene and how it should evolve over time.*
004 *We investigate a decoupled framework inspired by the Joint-*
005 *Embedding Predictive Architecture (JEPA): a causal Trans-*
006 *former (Flow Model) predicts the trajectory of CLIP em-*
007 *beddings in a compact 512-dimensional semantic space,*
008 *and a conditioned UNet (Renderer) converts each predicted*
009 *embedding back to a pixel frame. We conduct a system-*
010 *atic set of experiments across four Renderer architectures—*
011 *FiLM conditioning, cross-attention to CLIP patch tokens,*
012 *contrastive conditioning loss, and optical flow prediction—*
013 *and find that each variant collapses to a mode where the*
014 *embedding condition is effectively ignored and the output*
015 *is a near-identity transformation of the input frame. We*
016 *present a detailed failure analysis identifying the primary*
017 *root cause: frozen CLIP ViT embeddings, trained for se-*
018 *mantic image-text alignment, lack the spatial configuration*
019 *and dense correspondence information necessary for pixel-*
020 *level frame rendering. Contributing factors—loss functions*
021 *that reward near-identity outputs and conditioning me-*
022 *chanisms unable to express spatial motion—are understood as*
023 *consequences of this information bottleneck. Despite the*
024 *negative result, our analysis clarifies the requirements for*
025 *embedding-conditioned video synthesis and suggests con-*
026 *crete alternative approaches.*

027 1. Introduction

028 Image-to-video (I2V) generation aims to produce a tempo-
029 rally coherent video sequence from a single static frame.
030 When conditioned on an action label (e.g., “walking”), the
031 generated video should depict natural motion consistent
032 with that action. This task is relevant to content creation,
033 video editing, and synthetic data augmentation for action
034 recognition systems.

035 Recent large-scale diffusion models such as Stable Video

Diffusion and Wan2.2 have demonstrated impressive results
by inflating image diffusion architectures into the tempo-
ral domain. However, these end-to-end approaches incur
substantial computational costs and remain prone to motion
flickering and identity drift.

Inspired by the JEPA framework, we investigate an al-
ternative: decoupling the problem into a **Flow Model** that
predicts motion in a semantic embedding space and a **Ren-**
derer that converts semantic predictions back to pixels. The
key hypothesis is that motion dynamics are easier to model
in a compact, abstract embedding space than in raw pix-
els. In an ideal scenario, the Flow Model would learn high-
level action dynamics (“a walking person alternates legs”),
and the Renderer would faithfully translate these semantic
blueprints into frames.

A central motivation for using frozen pre-trained en-
coders (CLIP ViT-B/32) was to investigate whether their
rich visual-semantic priors could reduce training costs.
Large-scale video generation models typically require enor-
mous datasets and compute budgets; if a frozen pre-trained
representation already captures meaningful visual structure,
a lightweight decoder might be sufficient to render motion
within that representation space. This is the core hypothe-
sis we set out to test: **can frozen pre-trained visual pri-**
ors serve as an effective backbone for low-cost image-to-
video generation?

We implement this framework using frozen CLIP ViT-
B/32 encoders, a causal Transformer for embedding trajec-
tory prediction, and a UNet-based Renderer with condition-
ing injected via FiLM, cross-attention, or explicit flow pre-
diction heads. We systematically ablate the conditioning
mechanism and loss function across four Renderer variants.

Our key finding is negative but instructive: across all
four variants, the Renderer learns to ignore the embedding
condition and produces near-identical outputs regardless of
input conditioning. The primary root cause, as our analysis
will show, is that frozen CLIP ViT embeddings—trained for
semantic alignment rather than spatial correspondence—
lack the information content necessary for pixel-level ren-
dering, a bottleneck that no conditioning architecture or loss

076	function could overcome.	
077	We present a detailed failure analysis quantifying the	
078	embedding signal strength, the collapse behavior across	
079	architectures, and the training dynamics. We believe	
080	these findings provide useful guidance for future work on	
081	embedding-conditioned generative models.	
082	2. Related Work	
083	2.1. Image-to-Video Generation	
084	Early video generation methods employed recurrent net-	
085	works or generative adversarial networks. More recently,	
086	diffusion-based approaches have become standard. Stable	
087	Video Diffusion fine-tunes a pre-trained image diffusion	
088	model on video data by adding temporal layers. Wan2.2 fol-	
089	lows a similar approach at larger scale. These models gener-	
090	ally operate in a VAE latent space, simultaneously modeling	
091	spatial structure and temporal dynamics in one pipeline. We	
092	experiment with a fundamentally different architecture that	
093	attempts to factor these responsibilities across two modules.	
094	2.2. Latent Predictive Architectures	
095	The JEPA framework proposes that predictive learning	
096	should operate in representation space rather than input	
097	space. V-JEPA applies this principle to self-supervised	
098	video learning by predicting masked spatio-temporal fea-	
099	tures. Our Flow Model follows the same intuition: pre-	
100	dict the trajectory of CLIP embeddings rather than raw pix-	
101	els. Where our work differs is the downstream rendering	
102	step—converting predicted embeddings back to pixels—	
103	which proves to be the critical bottleneck.	
104	2.3. Conditioned Image Generation	
105	Feature-wise Linear Modulation (FiLM) and cross-	
106	attention are standard techniques for conditioning image	
107	generation on external information. FiLM applies channel-	
108	wise affine transformations $\gamma(c) \odot x + \beta(c)$ predicted from	
109	a conditioning vector c , and has been widely used in vi-	
110	sual question answering, style transfer, and video predic-	
111	tion. Cross-attention, popularized by the Transformer ar-	
112	chitecture, allows each spatial position in a feature map to	
113	attend to a set of conditioning tokens. Our experiments test	
114	both mechanisms independently and in combination, and	
115	find that neither is sufficient when the conditioning signal	
116	(adjacent-frame CLIP embeddings) varies by less than 3%	
117	in cosine distance.	
118	2.4. Optical Flow and Warping	
119	Frame interpolation and video prediction often employ ex-	
120	PLICIT optical flow prediction followed by warping. Methods	
121	such as RAFT achieve state-of-the-art optical flow estima-	
122	tion through iterative refinement, while simpler approaches	
123	use Farneback or TV-L1 flow as a supervision signal. Our	
	final Renderer variant adopts a flow-based approach: a	124
	UNet backbone predicts a dense flow field conditioned on	125
	CLIP embeddings, and the previous frame is warped via	126
	differentiable bilinear sampling. We use Farneback optical	127
	flow as pseudo-supervision and apply total variation regu-	128
	larization to encourage spatial smoothness.	129
	2.5. Action-Conditioned Video Generation	130
	Motion generation conditioned on action labels has been	131
	studied in controlled settings such as video game environ-	132
	ments and simplified human motion benchmarks. Text-to-	133
	video models such as Make-A-Video and Imagen Video	134
	use large language model embeddings for open-vocabulary	135
	conditioning at scale. Our setting is intermediate: we use	136
	closed-set action labels encoded through CLIP text embed-	137
	dings, providing a clean semantic signal in the same embed-	138
	ding space as the visual features.	139
	3. Data	140
	We use the HMDB51 dataset, containing 6,754 video	141
	clips across 51 human action categories such as “walking,”	142
	“drinking,” “brushing hair,” and “climbing stairs.” Videos	143
	average approximately 3 seconds in duration at 25–30 fps.	144
	All available videos (6,754) are used for training; metrics	145
	reported in this paper are computed on the training set, as	146
	our investigation focused on training dynamics and con-	147
	ditioning sensitivity rather than generalization to held-out	148
	data.	149
	3.1. Preprocessing	150
	For each video, we sample 17 consecutive frames ($n = 16$	151
	prediction steps) at a stride of 2, and resize all frames to	152
	224×224 pixels. CLIP embeddings (CLS token + 49 patch	153
	tokens) are pre-extracted for the entire dataset (242MB) to	154
	avoid redundant forward passes during Flow Model train-	155
	ing. The Renderer samples random frame pairs (I_{t-1}, I_t)	156
	from each video with a gap of 1 frame.	157
	3.2. Embedding Analysis	158
	A key diagnostic we perform early is measuring the varia-	159
	tion in CLIP embeddings across frames. Table 1 summa-	160
	rizes pairwise cosine similarities.	161
	The critical observation is that adjacent-frame embed-	162
	dings differ by only $1 - \cos \approx 0.024$ on average. This means	163
	the embedding trajectory that the Flow Model must predict,	164
	and that the Renderer must use as a conditioning signal,	165
	contains very little per-frame variation. Any learning algo-	166
	rithm must amplify this tiny signal by orders of magnitude	167
	to produce meaningful pixel-space changes.	168
	4. Methods	169
	Our framework consists of three frozen encoders and two	170
	trainable modules. The pipeline is as follows: (1) a frozen	171

Table 1. CLIP ViT-B/32 embedding cosine similarity. Same-video frames are highly similar ($1 - \text{cos} = 0.024$ for adjacent frames), limiting the information available for frame-to-frame motion prediction.

Comparison	Mean cos
Same video, distant frames ($\Delta t \approx 60$)	0.862
Same video, adjacent frames ($\Delta t = 1$)	0.976
Different videos, same action class	0.664
Different action classes	0.564

CLIP ViT encodes the input image I_0 to E_0 and extracts spatial patch tokens; (2) a frozen CLIP text encoder produces the label embedding L ; (3) the Flow Model—a causal Transformer—predicts the embedding trajectory $(\hat{E}_1, \dots, \hat{E}_n)$ from (L, E_0) using shifted teacher forcing; (4) the Renderer—a UNet with FiLM and cross-attention conditioning—produces each frame I_t from I_{t-1} , conditioned on $\hat{E}_t$, L , and patch tokens. The two models are trained separately.

4.1. Encoders

Image Encoder. Frozen CLIP ViT-B/32. We extract both the 512-dim CLS embedding E_t and the 49 spatial patch tokens (7×7 grid, 512-dim each) from the final vision transformer layer.

Label Encoder. Frozen CLIP text encoder. Action labels are converted from directory names to natural text (e.g., “brush hair”) and encoded to 512-dim semantic embeddings L .

4.2. Flow Model

We initially attempted a continuous flow matching formulation: learning a velocity field $v_\theta(E_t, t, L)$ that transports E_0 to E_n through the embedding space. This approach produced unstable trajectories and poor convergence, which motivated our switch to a discrete autoregressive formulation.

Our final Flow Model is a 4-layer causal Transformer encoder (12.9M parameters) with 8 attention heads and feedforward dimension 2048. The input sequence is $[L, E_0, E_1, \dots, E_{n-1}]$, and the model is trained with shifted teacher forcing: position t predicts $\hat{E}_t$ from $(L, E_0, \dots, E_{t-1})$.

We compare two loss functions:

- **MSE:** $\mathcal{L} = \frac{1}{n} \sum_t \|\hat{E}_t - E_t\|_2^2$
- **Cosine:** $\mathcal{L} = \frac{1}{n} \sum_t (1 - \cos(\hat{E}_t, E_t))$

4.3. Renderer

The Renderer is a UNet (4 down/up blocks, 40–43M parameters depending on variant) that takes I_{t-1} as input and

produces an output conditioned on $\hat{E}_t$, L , and patch tokens. We experiment with four variants:

1. **FiLM (baseline):** channel-wise Feature-wise Linear Modulation from a condition vector $[\hat{E}_t; L] \in \mathbb{R}^{1024}$, applied at each residual block. Output: pixel residual ΔI_t . Loss: $\text{L1}(\Delta I_t, I_t - I_{t-1})$.
2. **FiLM + Cross-Attention:** FiLM conditioning plus multi-head cross-attention from UNet feature maps to CLIP patch tokens at the bottleneck and first two upsampling blocks. Output: pixel residual ΔI_t . Loss: L1.
3. **FiLM + Cross-Attention + Contrastive:** same as variant 2, plus a contrastive conditioning loss: for a batch, we compute $\mathcal{L}_{\text{cont}} = \|\cos(\Delta I_{\text{real}}, \Delta I_{\text{fake}}) - \cos(E_{\text{real}}, E_{\text{fake}})\|_2^2$, where fake conditions are shuffled embeddings from other videos in the batch. This explicitly penalizes the Renderer for producing identical outputs from different conditions.
4. **Flow Prediction:** same UNet backbone with FiLM + cross-attention, but the output head predicts a dense optical flow field $\in \mathbb{R}^{2 \times 224 \times 224}$. The previous frame is warped via the predicted flow: $I_t = \text{warp}(I_{t-1}, \text{flow})$. Loss: $\text{L1}(I_t^{\text{warped}}, I_t^{\text{gt}}) + \lambda_{\text{TV}} \cdot \text{TV}(\text{flow}) + \lambda_{\text{FB}} \cdot \text{L1}(\text{flow}, \text{Farneback})$.

4.4. Training Details

All models are trained with AdamW ($lr = 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 10^{-2}) and a cosine annealing learning rate schedule over 100 epochs. We use a batch size of 8 with gradient accumulation over 2 steps (effective batch size 16). Gradient clipping with maximum norm 1.0 is applied. All experiments were conducted on a single NVIDIA RTX 4060 Laptop GPU (8GB VRAM). Complete training of the Flow Model takes approximately 4 hours; each Renderer variant takes approximately 7 hours.

Flow Model training data pipeline. Because passing CUDA models through PyTorch DataLoader workers is not supported on Windows, we pre-extract CLIP embeddings for all 6,754 videos (17 frames each) and save them to disk as a single 242MB file. The Flow Model training loop loads pre-extracted embeddings directly, requiring no CLIP forward passes during training. This eliminates the DataLoader CUDA issue and accelerates training by approximately $3 \times$.

Renderer training data pipeline. The Renderer uses HMDB51FrameDataset, which loads raw frames from disk (via OpenCV) and performs image transformations in the DataLoader workers. CLIP encoding happens in the main training loop where CUDA is available. Farneback optical flow is computed on CPU using OpenCV, adding approximately 30% overhead per batch.

4.5. Implementation Notes

Shifted teacher forcing. An earlier version of the Flow Model used “zero-placeholder” inputs where the model received $[L, E_0, \mathbf{0}, \dots, \mathbf{0}]$ and was expected to predict all future embeddings in a single forward pass. This created a train-inference mismatch: at inference time, the model sees its own predictions rather than zeros. Shifted teacher forcing, where the model receives the real embedding sequence and predicts one step ahead at each position, aligns training and inference exactly. We found this improved embedding trajectory quality significantly (from 2.0% to 82.5% of GT motion, combined with the cosine loss change).

Multi-scale flow prediction. In the flow prediction Renderer variant, we considered predicting flow at multiple decoder resolutions and combining them via coarse-to-fine refinement (similar to RAFT). However, initial experiments showed that even a single-scale flow head struggled to produce consistent non-zero outputs, so we deferred multi-scale investigation to future work.

5. Experiments

5.1. Evaluation Protocol

We evaluate using the following metrics:

- **Embedding Trajectory MSE:** MSE between predicted and ground-truth CLIP embedding sequences.
- **Inter-frame 1-cos Distance:** $1 - \cos(\hat{E}_t, \hat{E}_{t+1})$, measuring how much “motion” the predicted embeddings contain.
- **Conditioning Sensitivity:** $\frac{\| \text{Renderer}(I, E_t) - \text{Renderer}(I, E_0) \|_1}{\| \text{Renderer}(I, E_0) \|_1 + \epsilon}$, measuring whether changing the embedding changes the Renderer output.

5.2. Flow Model Results

Table 2 summarizes the Flow Model results.

Table 2. Flow Model performance with MSE vs. cosine similarity loss. Cosine loss prevents embedding collapse while maintaining prediction accuracy.

Metric	MSE Loss	Cosine Loss
Train loss (final)	0.0116	0.0264
Inter-frame 1-cos (GT)	0.0290	0.0290
Inter-frame 1-cos (pred)	0.0006	0.0239
% of GT motion preserved	2.0%	82.5%

The MSE-trained Flow Model achieves low MSE (0.0116) but collapses to predicting nearly identical embeddings at every timestep (1-cos $\approx$ 0.0006). Cosine loss resolves this: predicted embeddings retain 82.5% of the ground-truth motion variation while maintaining comparable embedding quality (MSE between predicted and GT embeddings actually improves in terms of cosine distance).

5.3. Renderer Results

Table 3 presents the core negative result: across all four architectures, the Renderer learns to ignore the embedding condition.

Table 3. Renderer conditioning sensitivity across four architecture variants. A ratio of ~ 1.0 indicates the model produces identical outputs regardless of the embedding condition.

Renderer Variant	Sensitivity Ratio
1. FiLM + L1	1.03x
2. FiLM + Cross-Attn + L1	0.98x
3. FiLM + Cross-Attn + L1 + Contrastive	0.98x
4. FiLM + Cross-Attn + Flow Pred + Rec + TV + FB	1.01x

For all variants, the conditioning sensitivity ratio is approximately 1.0, meaning the magnitude of the Renderer’s output does not meaningfully change when the conditioning embedding changes. In variants 1–3 (residual prediction), the absolute output magnitude converges to ≈ 0.005 —essentially near-zero. In variant 4 (flow prediction), the predicted flow field converges back to zero after initial non-zero activity from random initialization.

5.4. Diagnostic Experiments

To isolate the failure mode, we conduct several diagnostic tests:

Embedding distance test. We feed the Renderer embeddings from frames at varying temporal distances ($[t, t + 1, t + 4, t + 8, t + 16]$) and measure the output magnitude. Even with frame $t + 16$ (1-cos = 0.31, a meaningful change), the output magnitude remains unchanged from the $t + 1$ case (1-cos = 0.03). This confirms that the model is not merely limited by the weak signal in adjacent frames—it has learned to ignore the embedding entirely.

Farneback flow as pseudo-target. We compute Farneback optical flow between adjacent frames (mean magnitude 0.39 px, max 2.84 px) and use it as a supervised target. The flow head initially produces non-zero flow but converges back to zero as the reconstruction and smoothness losses dominate.

CLIP feature discrimination. We verify that CLIP embeddings *do* contain frame-to-frame information: the cosine similarity gap between same-video adjacent frames (0.976) and different videos of the same class (0.664) is 0.312—a substantial difference. The information exists, but translating it from a 512-dim global vector into spatially meaningful pixel changes proves fundamentally difficult with our architectures.

334	6. Discussion: Why Did It Fail?	
335	We identify the following contributing factors, ordered from	
336	most fundamental to most specific.	
337	6.1. Primary Cause: Frozen CLIP Embeddings Are	
338	Unsuitable for This Task	
339	We believe this is the root cause from which all other issues	
340	derive. Several lines of evidence point specifically to the	
341	frozen pre-trained ViT embeddings—rather than the Trans-	
342	former architecture or the decoupled pipeline design—as	
343	the bottleneck.	
344	First, purely autoregressive Transformer-based video	
345	generation has been demonstrated successfully in prior	
346	work (e.g., VideoGPT, TATS, and more recent video tok-	
347	enizers combined with autoregressive transformers). These	
348	models generate videos token-by-token or frame-by-frame,	
349	proving that the autoregressive Transformer paradigm is not	
350	inherently flawed for video generation. The key difference	
351	is that these models operate on <i>learned</i> discrete tokens from	
352	a VQ-VAE trained jointly for reconstruction, whereas our	
353	Flow Model operates on <i>frozen</i> CLIP embeddings that were	
354	never optimized for the rendering task.	
355	Second, CLIP ViT-B/32 is trained with a contrastive	
356	image-text matching objective: given an image and a text	
357	description, maximize the cosine similarity of their embed-	
358	dings. This objective encourages the model to extract <i>se-</i>	
359	<i>mantic content</i> —“what is in this image”—and compresses	
360	it into a compact 512-dimensional vector. What CLIP’s	
361	training objective does <i>not</i> require is preserving the kind	
362	of information needed for frame-to-frame rendering:	
363	• Spatial configuration: The precise 2D coordinates of	
364	body parts, objects, and their relative positions.	
365	• Pose and articulation: The joint angles of limbs, the ori-	
366	entation of a head, the bend of a knee.	
367	• Dense correspondence: Which pixel in frame t corre-	
368	sponds to which pixel in frame $t + 1$.	
369	A CLIP embedding can tell you “this is a person walk-	
370	ing” with high confidence, but it cannot tell you “the right	
371	leg is currently forward at a 30-degree angle, positioned at	
372	pixel coordinates (120, 180).” The latter is precisely the in-	
373	formation a Renderer needs to predict what the next frame	
374	should look like.	
375	This explains the asymmetry in our results: the Flow	
376	Model succeeded because predicting an embedding trajec-	
377	tory only requires matching the statistical distribution of	
378	CLIP embeddings—a task CLIP embeddings contain suf-	
379	ficient information for. The Renderer failed because de-	
380	coding an embedding back to a specific spatial configura-	
381	tion of pixels requires information that was discarded dur-	
382	ing CLIP’s encoding process.	
383	Empirically, we confirmed this information bottleneck:	
384	adjacent frames of the same video have CLIP embeddings	
	with cosine similarity ≈ 0.976 ($1 - \cos \approx 0.024$). This	385
	tiny difference must encode the entire motion transforma-	386
	tion from one frame to the next—a compression ratio far	387
	exceeding what any decoder architecture can plausibly in-	388
	vert.	389
	6.2. Contributing Factor: Loss Function Misalign-	390
	ment	391
	All our loss functions (L1 on residuals, L1 on reconstruc-	392
	tion after warping) have an optimal trivial solution near the	393
	identity transformation: $\Delta I_t \approx 0$ and flow ≈ 0 . When the	394
	conditioning signal carries almost no usable information (as	395
	argued above), identity mapping becomes the rational opti-	396
	mum.	397
	6.3. Contributing Factor: FiLM Cannot Express	398
	Spatial Guidance	399
	Feature-wise Linear Modulation applies channel-wise scal-	400
	ing and shifting uniform across all spatial positions. Even if	401
	the embedding <i>did</i> contain spatial information, FiLM lacks	402
	the mechanism to route it to the correct spatial location.	403
	Cross-attention to CLIP patch tokens partially addresses	404
	this routing problem, but the patch tokens themselves still	405
	lack dense correspondence information due to the primary	406
	cause above.	407
	6.4. Limitations of Our Study	408
	We acknowledge several limitations: (1) we only explored	409
	CLIP ViT-B/32 as the embedding backbone—larger ViT	410
	variants or dense-prediction models like DINOv2 may have	411
	different properties; (2) we did not attempt joint end-to-end	412
	training of the Flow Model and Renderer, which could al-	413
	low the Flow Model to learn embeddings specifically opti-	414
	mized for rendering; (3) we used a relatively small dataset	415
	(6.7k videos), and the collapse behavior may differ at larger	416
	scale where more diverse motion patterns provide a stronger	417
	learning signal; (4) HMDB51 videos are constrained to spe-	418
	cific human action categories, and the learned dynamics	419
	may not generalize to more open-ended video generation.	420
	6.5. What We Learned	421
	Despite the negative result, this investigation yielded several	422
	lessons that may benefit future work in this area:	423
	Embedding choice is critical. The choice of CLIP ViT-	424
	B/32 as our embedding backbone was motivated by its joint	425
	vision-language space and compact dimensionality. How-	426
	ever, CLIP embeddings are optimized for semantic dis-	427
	crimination, not spatial correspondence. For a decoupled	428
	I2V pipeline to succeed, the embedding space must encode	429
	both <i>what</i> is in the scene and <i>where</i> things are spatially—	430
	properties that CLIP’s contrastive training objective does	431
	not guarantee.	432

Loss functions define the optimization landscape. We observed that all tested pixel-space loss functions (L1, L1 + TV, L1 + Farneback) contain a near-identity trivial solution ($\Delta I \approx 0$, flow ≈ 0). This is not specific to our architecture—any model trained with a pixel reconstruction loss on adjacent-frame pairs faces the same challenge. Loss functions that operate in a learned feature space (perceptual loss, adversarial loss) may provide a stronger signal for motion learning.

The JEPA philosophy needs a rendering pathway. JEPA’s core insight—predict in representation space, not input space—proved effective for our Flow Model (82.5% of GT motion). But the companion challenge of *decoding* those predictions back to the pixel domain is equally hard and received comparatively little attention in our implementation. The rendering problem deserves its own careful treatment rather than being treated as an architectural afterthought.

6.6. Future Work

Several promising directions emerge from this analysis.

Dense spatial embeddings. Replacing CLIP with a model trained for dense visual correspondence—such as DINOv2, Stable Diffusion’s VAE latents, or a self-supervised optical flow network—could provide embeddings that naturally encode spatial configuration. A trajectory in DINOv2 patch feature space would carry richer spatial information than a trajectory in CLIP CLS space.

Joint end-to-end training. Training the Flow Model and Renderer jointly, with gradients flowing from the rendering loss back through the Flow Model, would allow the embedding space to adapt to the needs of the Renderer. The Flow Model would no longer be constrained to match CLIP’s embedding structure and could learn features specifically optimized for frame rendering.

Iterative refinement. Rather than predicting the full frame transformation in a single forward pass, a multi-step refinement process (similar to diffusion models or recurrent refinement) could gradually build up the output from an initial near-identity estimate. This would allow the model to learn motion in stages—coarse structure first, then fine details.

Adversarial and perceptual losses. GAN-based losses or CLIP-based perceptual losses directly penalize lack of motion and unrealistic static output, potentially breaking the near-identity local minimum that trapped all of our L1-trained models.

Explicit motion field supervision. Using a pre-trained optical flow network (e.g., RAFT) instead of Farneback as a supervision target could provide a cleaner learning signal. Combined with a rendering loss, this could guide the model toward physically plausible motion while maintaining visual fidelity.

7. Conclusion

We set out to build a decoupled image-to-video generation system in which a causal Transformer predicts action-conditioned embedding trajectories and a conditioned UNet renders those trajectories to pixels. The Flow Model component was partially successful: with cosine similarity loss it learned to predict embedding trajectories preserving 82.5% of ground-truth motion variation, while MSE loss led to complete collapse. However, the Renderer failed across all four conditioning architectures tested: FiLM, cross-attention, contrastive conditioning, and optical flow prediction.

Our analysis suggests the primary cause is a fundamental information bottleneck: CLIP ViT-B/32 embeddings are trained for semantic image-text alignment and do not preserve the spatial configuration, pose, and correspondence information necessary for pixel-level frame rendering. Adjacent frames differ by less than 3% in CLIP embedding cosine distance, an information compression ratio that is implausible for any decoder architecture to invert. All other observed problems—loss function collapse to identity, FiLM’s inability to express spatial motion—can be understood as consequences of this root cause.

Returning to our original research question—whether frozen pre-trained visual priors can serve as an effective backbone for low-cost I2V generation—our results suggest a nuanced answer. The priors *do* carry useful semantic information: our Flow Model successfully learned action-conditioned embedding trajectories with 82.5% of ground-truth motion variation, and the embedding space cleanly separates different action classes (inter-class 1-cos ≈ 0.44 vs. intra-class ≈ 0.34). However, semantic priors alone are insufficient for the *rendering* stage, which requires fine-grained spatial information that CLIP’s training objective discards. The lesson is not that pre-trained priors are useless for video generation, but that the choice of pre-trained representation must be matched to *both* stages of a decoupled pipeline—the prediction stage needs semantic structure, while the rendering stage needs spatial correspondence.

Future directions suggested by this analysis include: (1) replacing CLIP with a dense visual representation trained for spatial correspondence, such as DINOv2 patch features, Stable Diffusion VAE latents, or self-supervised optical flow features; (2) training the Flow Model and Renderer jointly end-to-end, so the embedding space can adapt to the specific needs of rendering rather than being constrained by a frozen pre-trained encoder; (3) adopting losses that explicitly penalize lack of motion, such as adversarial or perceptual losses, to break the near-identity local minimum; (4) incorporating explicit motion representations such as optical flow or keypoint trajectories as intermediate supervision targets rather than relying solely on the embed-

ding to carry all motion information.

References

[1] A. Blattmann, T. Dockhorn, S. Kulal, et al. “Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets.” *arXiv:2311.15127*, 2023.

[2] Alibaba. “Wan2.2: Large-Scale Image-to-Video Generation.” 2025–2026.

[3] Y. LeCun. “A Path Towards Autonomous Machine Intelligence.” *OpenReview*, 2022.

[4] A. Bardes, Q. Garrido, J. Ponce, et al. “Revisiting Feature Prediction for Learning Visual Representations from Video.” *arXiv:2404.08471*, 2024.

[5] A. Radford, J. W. Kim, C. Hallacy, et al. “Learning Transferable Visual Models From Natural Language Supervision.” *ICML*, 2021.

[6] H. Kuehne, H. Jhuang, E. Garrote, et al. “HMDB: A Large Video Database for Human Motion Recognition.” *ICCV*, 2011.

[7] E. Perez, F. Strub, H. de Vries, et al. “FiLM: Visual Reasoning with a General Conditioning Layer.” *AAAI*, 2018.

[8] R. Zhang, P. Isola, A. A. Efros, et al. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric.” *CVPR*, 2018.

[9] A. Vaswani, N. Shazeer, N. Parmar, et al. “Attention Is All You Need.” *NeurIPS*, 2017.

[10] P. Isola, J.-Y. Zhu, T. Zhou, et al. “Image-to-Image Translation with Conditional Adversarial Networks.” *CVPR*, 2017.

[11] O. Ronneberger, P. Fischer, T. Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation.” *MICCAI*, 2015.

[12] J. Oh, X. Guo, H. Lee, et al. “Action-Conditional Video Prediction using Deep Networks in Atari Games.” *NIPS*, 2015.

[13] N. Srivastava, E. Mansimov, R. Salakhutdinov. “Unsupervised Learning of Video Representations using LSTMs.” *ICML*, 2015.

[14] C. Vondrick, H. Pirsiavash, A. Torralba. “Generating Videos with Scene Dynamics.” *NIPS*, 2016.

[15] U. Singer, A. Polyak, T. Hayes, et al. “Make-A-Video: Text-to-Video Generation without Text-Video Data.” *arXiv:2209.14792*, 2022.

[16] J. Ho, W. Chan, C. Saharia, et al. “Imagen Video: High Definition Video Generation with Diffusion Models.” *arXiv:2210.02303*, 2022.

[17] M. Monfort, A. Andonian, B. Zhou, et al. “Moments in Time Dataset: One Million Videos for Event Understanding.” *TPAMI*, 2019.